कंप्यूटर विज्ञान एवं अभियांत्रिकी विभाग
Department of Computer Science and Engineering
हरियाणा केन्द्रीय विश्वविद्यालय
Central University of Haryana
जांट--पाली, महेंद्रगढ़--१२३०३१
Jant-Pali, Mahendergarh, Haryana-123031.



Lab Report

of

<Subject Name in All Caps>

(<Subject Code>)

Submitted by:

Name: <Student Name>
Roll No: <Student Roll Number>

Submitted to:

Mr. Anant R. Bara
(Assistant Professor)
Department of CSE

INDEX

Program 1: WAP to demonstrate Dynamic Command-Line Caesar Cipher Tool. 4
Program 2: WAP to detect : Automated Word Frequency and Pattern Analyzer. 6
Program 3: WAP for Automated CSV File Inspector and Data Exporter. 9
Program 4:
Write a program for interactive command line scheduler with lazy evaluation 12
0.0.1 Output 13
Program 5: Write a program for Robust file management and directory navigation utility. 15
Program 6:
Write a program for creating a dynamic phonebook and contact directory manager. 18
0.0.2 Output 19
Program 7:
Write a program for functional data filtering and matrix transformation suite. 21
0.0.3 Output 22
Program 8:
Write a program for Custom Flexible Student Grade and Statistics Calculator. 23
0.0.4 Output 24
Program 9:
Write a program for making Text Anagram and Pattern Matcher Tool. 25
0.0.5 Output 26
Program 10:
Write a program to manage In-Memory Inventory Tracker with Dictionary Merging. 27
0.0.6 Output 27
Program 11:
Write a program for Interactive Turtle Pattern Generator and Fractal Renderer. 29
0.0.7 Output 30
Program 12:
Write a program for Digital Image Filter and Color Converter Utility. 31
0.0.8 Output 31

Program 13:
Write a program to Encapsulated Banking System with Inheritance Models. 33
0.0.9 Output 34
Program 14:
Write a program for Object Persistence Engine via Serialization. 35
0.0.10 Output 36
Program 15:
Write a program to Abstract Shape Math Engine with Operator Overloading. 37
0.0.11 Output 38
Program 16:
Write a program to create a Safe File Processor with Custom Exception Hierarchy. 39
0.0.12 Output 40
Program 17:
Write a program to create a Multi-Functional Graphical Desktop Calculator. 41
0.0.13 Output 42
Program 18:
Write a program to create a GUI-based student record manager that stores student details in a file. 44
0.0.14 Output 46
Program 19:
Write a program to create a GUI application that allows the user to select and view image or text files using dialog boxes. . . 47
0.0.15 Output 48
Program 20:
Write a program to create a modular multi-frame GUI dashboard using Tkinter, menus, buttons and frames. 50
0.0.16 Output 53

List of Figures

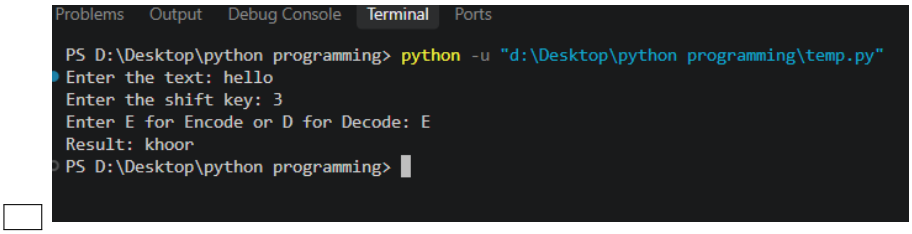
1 Output of Program 4 13
2 Output of Program 6 19
3 Output of Program 7 22
4 Output of Program 8 24
5 Output of Program 9 26
6 Output of Program 10 27
7 Output of Program 11 30
8 Output of Program 12 31
9 Output of Program 13 34
10 Output of Program 14 36
11 Output of Program 15 38
12 Output of Program 16 40
13 Output of Program 17 42
14 Output of Program 18 46
15 Output of Program 19 48
16 Output of Program 20 53

Program 1: WAP to demonstrate Dynamic Command-Line Caesar Cipher Tool.

Code

```
1
2  import sys
3
4  def caesar_cipher(text, shift):
5      result = ""
6
7      for char in text:
8          if char.isalpha():
9              if char.isupper():
10                 result += chr((ord(char) - ord('A') + shift) % 26 +
       ord('A'))
11
12             else:
13                 result += chr((ord(char) - ord('a') + shift) % 26 +
       ord('a'))
14         else:
15             result += char
16
17     return result
18
19 if len(sys.argv) >= 3:
20     text = sys.argv[1]
21     shift = sys.argv[2]
22
23 else:
24     text = input("Enter the text: ")
25     shift = input("Enter the shift key: ")
26
27
28 if not shift.lstrip("-").isdigit():
29     print("Error: Shift key must be an integer.")
30     sys.exit(1)
31
32 shift = int(shift)
33
34 choice = input("Enter E for Encode or D for Decode: ").upper()
35
36 if choice == "E":
37     result = caesar_cipher(text, shift)
38
39 elif choice == "D":
40     result = caesar_cipher(text, -shift)
41
42 else:
43     print("Error: Please enter E or D.")
44     sys.exit(1)
45
46 print("Result:", result)
```

Output



GitHub Scan Code

Scan to view source code
[cite: 49]

Program 2: WAP to detect : Automated Word Frequency and Pattern Analyzer.

Code

```
import string

def analyze_text(text):
    # Clean extra whitespace
    text = " ".join(text.split())

    # Remove punctuation
    cleaned_text = text.translate(str.maketrans("", "", string.
    punctuation))

    # Convert to lowercase
    words = cleaned_text.lower().split()

    # Total word count
    total_words = len(words)

    # Frequency table
    frequency = {}

    for word in words:
        frequency[word] = frequency.get(word, 0) + 1

    # Find palindrome words
    palindromes = []

    for word in words:
        if len(word) > 1 and word == word[::-1]:
            if word not in palindromes:
                palindromes.append(word)

    # Display report
    print("\n========== TEXT ANALYSIS REPORT ==========")
    print("Cleaned Text:", cleaned_text)
    print("Total Words:", total_words)

    print("\nWord Frequency:")
    for word, count in sorted(frequency.items()):
        print(f"{word}: {count}")

    print("\nPalindrome Words:")
    if palindromes:
        print(", ".join(palindromes))
    else:
        print("No palindromes found.")


# Choose input method
print("1. Enter multiline text")
print("2. Read text from a file")

choice = input("Enter your choice (1/2): ")

```

```python
if choice == "1":
    print("\nEnter your text.")
    print("Type END on a new line to finish:")

    lines = []

    while True:
        line = input()
        if line == "END":
            break
        lines.append(line)

    text = "\n".join(lines)
    analyze_text(text)

elif choice == "2":
    filename = input("Enter file name: ")

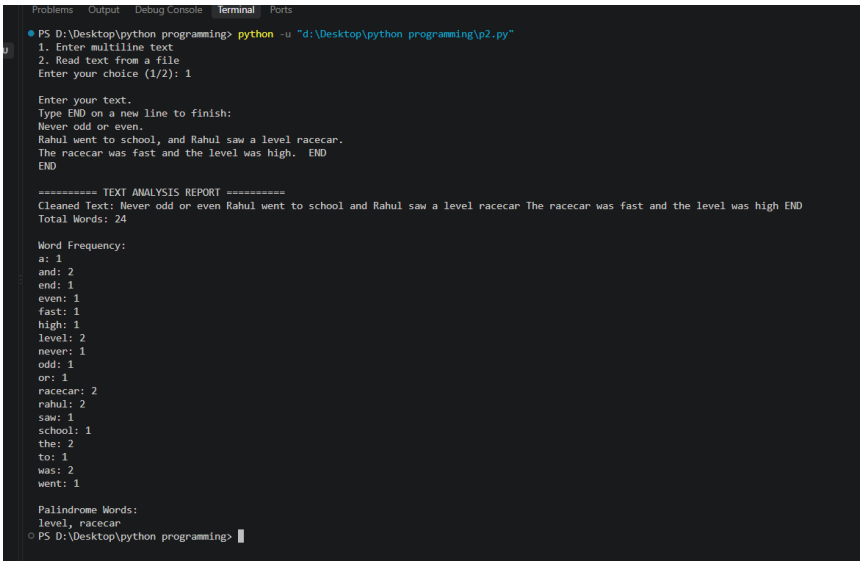
    try:
        with open(filename, "r", encoding="utf-8") as file:
            text = file.read()

        analyze_text(text)

    except FileNotFoundError:
        print("Error: File not found.")

else:
    print("Invalid choice.")
```

Output



GitHub Scan Code



Scan to view source code
[cite: 49]

Program 3: WAP for Automated CSV File Inspector and Data Exporter.

Code

```
1
2  import pandas as pd
3  import os
4
5  print("===== CSV FILE INSPECTOR AND DATA EXPORTER =====")
6
7  # Take file name from user
8  filename = input("Enter CSV/TSV file name: ")
9
10 # Check whether file exists
11 if not os.path.exists(filename):
12     print("Error: File not found!")
13     exit()
14
15 # Detect separator
16 if filename.endswith(".tsv"):
17     separator = "\t"
18 else:
19     separator = ","
20
21 # Read the file
22 df = pd.read_csv(filename, sep=separator)
23
24 # Display basic information
25 print("\n===== DATASET INFORMATION =====")
26 print("Number of Rows:", df.shape[0])
27 print("Number of Columns:", df.shape[1])
28
29 print("\nColumn Names:")
30 print(list(df.columns))
31
32 print("\nData Types:")
33 print(df.dtypes)
34
35 print("\nFirst 5 Rows:")
36 print(df.head())
37
38 print("\n===== SUMMARY STATISTICS =====")
39 print(df.describe())
40
41 # Select column for filtering
42 column = input("\nEnter column name for filtering: ")
43
44 if column in df.columns:
45
46     # Try numeric filtering
47     if pd.api.types.is_numeric_dtype(df[column]):
48
49         value = float(input("Enter minimum value: "))
50
51         filtered_data = df[df[column] >= value]
52
```

```
        print("\n===== FILTERED DATA =====")
        print(filtered_data)

    else:
        # String filtering
        value = input("Enter value to search: ")

        filtered_data = df[
            df[column].astype(str).str.contains(value, case=False, na=False)
        ]

        print("\n===== FILTERED DATA =====")
        print(filtered_data)

    # Export filtered data
    output_file = input("\nEnter output file name (example: cleaned.csv): ")

    filtered_data.to_csv(output_file, index=False)

    print("\nFiltered data saved successfully!")
    print("Output file:", output_file)

else:
    print("Error: Column not found!")student.cs
```

Output

```
PS D:\Desktop\python programming> python -u "d:\Desktop\python programming\p3.py"
===== CSV FILE INSPECTOR AND DATA EXPORTER =====
Enter CSV/TSV file name: student.csv

===== DATASET INFORMATION =====
Number of Rows: 8
Number of Columns: 4

Column Names:
['Name', 'Age', 'Marks', 'City']

Data Types:
Name       str
Age      int64
Marks    int64
City       str
dtype: object

First 5 Rows:
    Name  Age  Marks      City
0  Rahul   20     85     Delhi
1   Aman   21     72  Dehradun
2  Priya   20     91     Delhi
3  Rohit   22     65  Haridwar
4   Neha   21     88  Dehradun

===== SUMMARY STATISTICS =====
             Age      Marks
count   8.000000   8.000000
mean   20.875000  80.125000
std     0.834523  11.089731
min    20.000000  65.000000
25%    20.000000  71.250000
50%    21.000000  80.500000
75%    21.250000  88.750000
max    22.000000  95.000000
```

GitHub Scan Code



Scan to view source code
[cite: 49]

Program 4:
Write a program for interactive command line scheduler with lazy evaluation

Code

```
import heapq

# Priority queue
tasks = []


# Generator for lazy task evaluation
def task_generator():
    while tasks:
        priority, task = heapq.heappop(tasks)
        yield priority, task


def add_task():
    task = input("Enter task: ")
    priority = int(input("Enter priority (1 = highest): "))

    heapq.heappush(tasks, (priority, task))
    print("Task added successfully!")


def view_tasks():
    if not tasks:
        print("No pending tasks.")
        return

    print("\nPending Tasks:")
    for priority, task in sorted(tasks):
        print(f"Priority: {priority} | Task: {task}")


def execute_task():
    if not tasks:
        print("No tasks to execute.")
        return

    # Lazy evaluation
    generator = task_generator()

    priority, task = next(generator)

    print(f"\nExecuting task: {task}")
    print(f"Priority: {priority}")


# Interactive command-line interface
while True:

    print("\n===== TASK SCHEDULER =====")
```

```
    print("1. Add Task")
    print("2. View Tasks")
    print("3. Execute Task")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        add_task()

    elif choice == "2":
        view_tasks()

    elif choice == "3":
        execute_task()

    elif choice == "4":
        print("Exiting Task Scheduler...")
        break

    else:
        print("Invalid choice. Try again.")
```

0.0.1 Output



Figure 1: Output of Program 4

GitHub Scan Code



Scan to view source code
[cite: 49]

Program 5: Write a program for Robust file management and directory navigation utility.

Code

```

import pandas as pd
import os

print("===== CSV FILE INSPECTOR AND DATA EXPORTER =====")

# Take file name from user
filename = input("Enter CSV/TSV file name: ")

# Check whether file exists
if not os.path.exists(filename):
    print("Error: File not found!")
    exit()

# Detect separator
if filename.endswith(".tsv"):
    separator = "\t"
else:
    separator = ","

# Read the file
df = pd.read_csv(filename, sep=separator)

# Display basic information
print("\n===== DATASET INFORMATION =====")
print("Number of Rows:", df.shape[0])
print("Number of Columns:", df.shape[1])

print("\nColumn Names:")
print(list(df.columns))

print("\nData Types:")
print(df.dtypes)

print("\nFirst 5 Rows:")
print(df.head())

print("\n===== SUMMARY STATISTICS =====")
print(df.describe())

# Select column for filtering
column = input("\nEnter column name for filtering: ")

if column in df.columns:

    # Try numeric filtering
    if pd.api.types.is_numeric_dtype(df[column]):

        value = float(input("Enter minimum value: "))

        filtered_data = df[df[column] >= value]

```

```
        print("\n===== FILTERED DATA =====")
        print(filtered_data)

    else:
        # String filtering
        value = input("Enter value to search: ")

        filtered_data = df[
            df[column].astype(str).str.contains(value, case=False, na=False)
        ]

        print("\n===== FILTERED DATA =====")
        print(filtered_data)

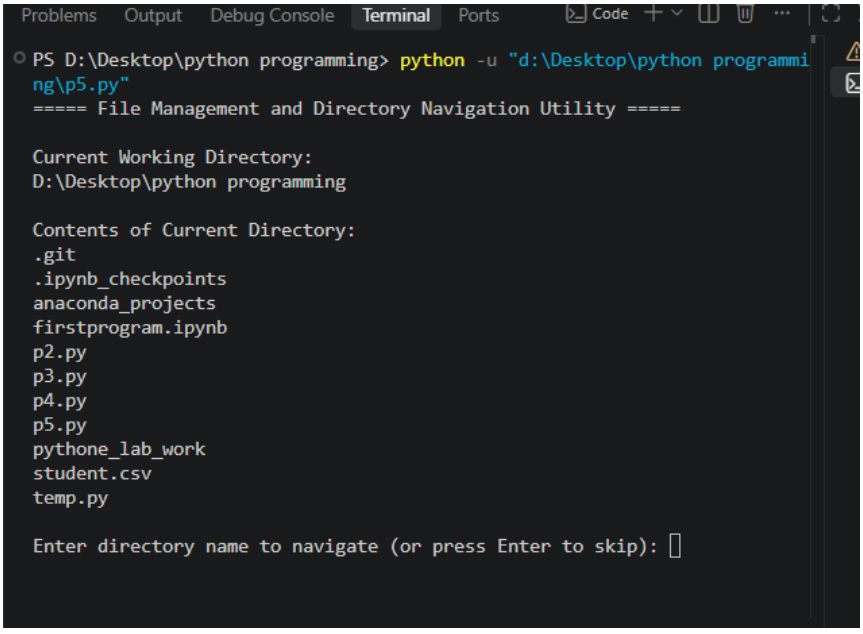
    # Export filtered data
    output_file = input("\nEnter output file name (example: cleaned.csv): ")

    filtered_data.to_csv(output_file, index=False)

    print("\nFiltered data saved successfully!")
    print("Output file:", output_file)

else:
    print("Error: Column not found!")student.cs
```

Output



GitHub Scan Code



Scan to view source code
[cite: 49]

Program 6:
Write a program for creating a dynamic phone-book and contact directory manager.

Code

```
1
2  # Program 6: Dynamic Phonebook and Contact Directory Manager
3
4  contacts = {}
5
6  while True:
7      print("\n===== PHONEBOOK MENU =====")
8      print("1. Add Contact")
9      print("2. Search Contact")
10     print("3. Update Contact")
11     print("4. Delete Contact")
12     print("5. Display All Contacts")
13     print("6. Exit")
14
15     choice = input("Enter your choice: ")
16
17     # Add Contact
18     if choice == "1":
19         name = input("Enter contact name: ")
20         phone = input("Enter phone number: ")
21
22         contacts[name] = phone
23         print("Contact added successfully.")
24
25     # Search Contact
26     elif choice == "2":
27         name = input("Enter name to search: ")
28
29         if name in contacts:
30             print("Phone Number:", contacts[name])
31         else:
32             print("Contact not found.")
33
34     # Update Contact
35     elif choice == "3":
36         name = input("Enter name to update: ")
37
38         if name in contacts:
39             phone = input("Enter new phone number: ")
40             contacts[name] = phone
41             print("Contact updated successfully.")
42         else:
43             print("Contact not found.")
44
45     # Delete Contact
46     elif choice == "4":
47         name = input("Enter name to delete: ")
48
49         if name in contacts:
50             del contacts[name]
```

```
            print("Contact deleted successfully.")
        else:
            print("Contact not found.")

    # Display All Contacts
    elif choice == "5":
        if len(contacts) == 0:
            print("Phonebook is empty.")
        else:
            print("\n===== ALL CONTACTS =====")
            for name, phone in contacts.items():
                print(name, ":", phone)

    # Exit
    elif choice == "6":
        print("Exiting Phonebook...")
        break

    else:
        print("Invalid choice. Please try again.")
```

0.0.2 Output



Figure 2: Output of Program 6

GitHub Scan Code

Scan to view source code
[cite: 49]

Program 7:
Write a program for functional data filtering and matrix transformation suite.

Code

```
1
2  # Program 7: Functional Data Filtering & Matrix Transformation
       Suite
3
4  # Input matrix
5  matrix = [
6      [5, 2, 8],
7      [7, 4, 1],
8      [9, 6, 3]
9  ]
10
11 print("Original Matrix:")
12 for row in matrix:
13     print(row)
14
15
16 # 1. Matrix Transformation
17 # Multiply every element by 2
18 transformed_matrix = [
19     list(map(lambda x: x * 2, row))
20     for row in matrix
21 ]
22
23 print("\nTransformed Matrix (Elements × 2):")
24 for row in transformed_matrix:
25     print(row)
26
27
28 # 2. Filtering Even Numbers
29 even_numbers = [
30     list(filter(lambda x: x % 2 == 0, row))
31     for row in matrix
32 ]
33
34 print("\nEven Numbers:")
35 for row in even_numbers:
36     print(row)
37
38
39 # 3. Flatten the matrix
40 data = [x for row in matrix for x in row]
41
42 # 4. Sort the elements
43 sorted_data = sorted(data)
44
45 print("\nSorted Data:")
46 print(sorted_data)
47
48
49 # 5. Sort in descending order
```

```python
descending_data = sorted(data, reverse=True)

print("\nSorted Data (Descending):")
print(descending_data)
```

0.0.3 Output



Figure 3: Output of Program 7

GitHub Scan Code

Scan to view source code
[cite: 49]

Program 8:
Write a program for Custom Flexible Student Grade and Statistics Calculator.

Code

```

def student_statistics(*marks, **details):

    minimum = min(marks)
    maximum = max(marks)
    average = sum(marks) / len(marks)

    grade_count = {
        "A": 0,
        "B": 0,
        "C": 0,
        "D": 0,
        "F": 0
    }

    for mark in marks:
        if mark >= 90:
            grade = "A"
        elif mark >= 80:
            grade = "B"
        elif mark >= 70:
            grade = "C"
        elif mark >= 60:
            grade = "D"
        else:
            grade = "F"

        grade_count[grade] += 1

    return minimum, maximum, average, grade_count, details


# Taking student details
name = input("Enter student name: ")

marks = list(map(int, input("Enter marks separated by spaces: ").
    split()))

minimum, maximum, average, grades, details = student_statistics(
    *marks, name=name
)

print("\nStudent Name:", details["name"])
print("Minimum Marks:", minimum)
print("Maximum Marks:", maximum)
print("Average Marks:", round(average, 2))

print("Grade Distribution:")
for grade, count in grades.items():
    print(grade, ":", count)
```

0.0.4 Output

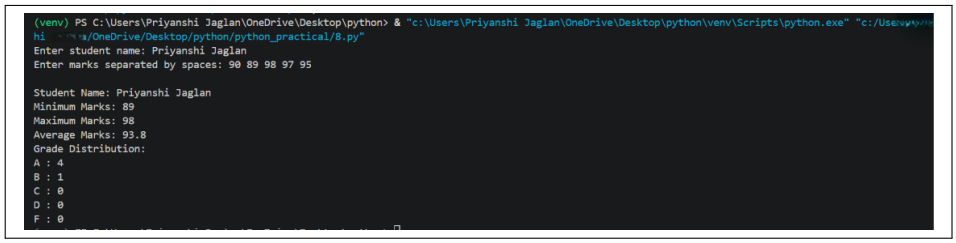


Figure 4: Output of Program 8

GitHub Scan Code

Scan to view source code
[cite: 49]

Program 9:
Write a program for making Text Anagram and Pattern Matcher Tool.

Code

```
1
2  def clean_text(text):
3      return "".join(text.lower().split())
4
5
6  def character_count(text):
7      count = {}
8
9      for ch in text:
10         count[ch] = count.get(ch, 0) + 1
11
12     return count
13
14
15 def anagram_check(text1, text2):
16
17     text1 = clean_text(text1)
18     text2 = clean_text(text2)
19
20     # Sorting method
21     if sorted(text1) == sorted(text2):
22         return True
23     else:
24         return False
25
26
27 def pattern(text):
28     count = character_count(clean_text(text))
29
30     # Convert dictionary into tuple
31     return tuple(sorted(count.items()))
32
33
34 # Input
35 text1 = input("Enter first word/phrase: ")
36 text2 = input("Enter second word/phrase: ")
37
38 if anagram_check(text1, text2):
39     print("The given texts are Anagrams.")
40 else:
41     print("The given texts are not Anagrams.")
42
43 # Dictionary counting
44 print("\nCharacter Pattern:")
45 print(pattern(text1))
```

0.0.5 Output



Figure 5: Output of Program 9

GitHub Scan Code

Scan to view source code
[cite: 49]

Program 10:
Write a program to manage In-Memory Inventory Tracker with Dictionary Merging.

Code

```
1
2  # Inventory of different stores
3
4  store1 = {
5      "Laptop": 10,
6      "Mouse": 20,
7      "Keyboard": 15
8  }
9
10 store2 = {
11     "Laptop": 5,
12     "Keyboard": 10,
13     "Monitor": 8
14 }
15
16 store3 = {
17     "Mouse": 10,
18     "Monitor": 5,
19     "Printer": 4
20 }
21
22
23 # Merging inventories
24 inventory = {}
25
26 for store in (store1, store2, store3):
27
28     for product, quantity in store.items():
29
30         inventory[product] = inventory.get(product, 0) + quantity
31
32
33 # Display final inventory
34 print("Combined Inventory:")
35
36 for product, quantity in inventory.items():
37     print(product, ":", quantity)
```

0.0.6 Output



Figure 6: Output of Program 10

GitHub Scan Code



Scan to view source code
[cite: 49]

Program 11:
Write a program for Interactive Turtle Pattern Generator and Fractal Renderer.

Code

```
1
2  import turtle
3
4  t = turtle.Turtle()
5  t.speed(0)
6  t.pensize(2)
7
8  # Draw polygon
9  sides = int(input("Enter number of sides: "))
10 length = int(input("Enter side length: "))
11
12 angle = 360 / sides
13
14 for i in range(sides):
15     t.forward(length)
16     t.right(angle)
17
18 # Koch curve
19 def koch(length, depth):
20     if depth == 0:
21         t.forward(length)
22     else:
23         length = length / 3
24
25         koch(length, depth - 1)
26         t.left(60)
27         koch(length, depth - 1)
28         t.right(120)
29         koch(length, depth - 1)
30         t.left(60)
31         koch(length, depth - 1)
32
33 # Move turtle to another position
34 t.penup()
35 t.goto(-150, -100)
36 t.pendown()
37
38 depth = int(input("Enter Koch fractal depth: "))
39
40 for i in range(3):
41     koch(300, depth)
42     t.right(120)
43
44 turtle.done()
```

0.0.7 Output

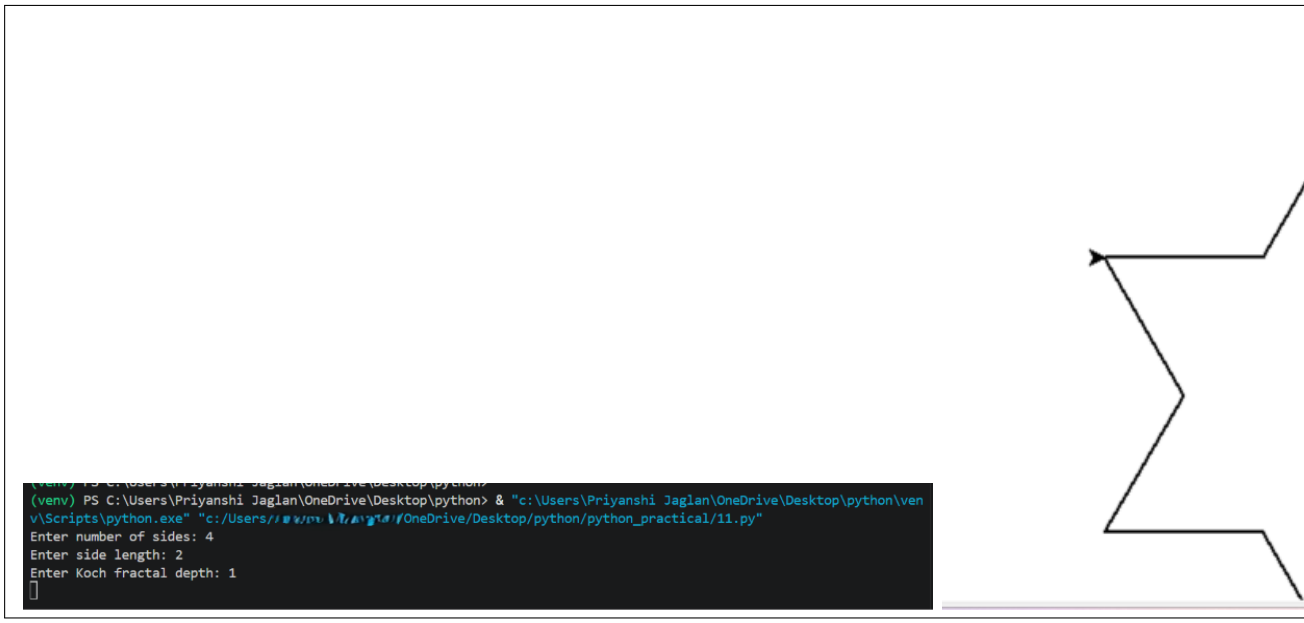


Figure 7: Output of Program 11

GitHub Scan Code

Scan to view source code
[cite: 49]

Program 12:
Write a program for Digital Image Filter and Color Converter Utility.

Code

```python

from PIL import Image, ImageFilter

filename = input("Enter image filename: ")

try:
    img = Image.open(filename)

    print("Image opened successfully.")
    print("Size:", img.size)

    # Grayscale
    gray = img.convert("L")
    gray.save("grayscale.png")

    # Black and white threshold
    bw = gray.point(lambda x: 255 if x > 128 else 0)
    bw.save("black_white.png")

    # Blur
    blur = img.filter(ImageFilter.BLUR)
    blur.save("blurred.png")

    print("Processed images saved successfully.")

except FileNotFoundError:
    print("Image file not found.")

except Exception as e:
    print("Error:", e)
```

0.0.8 Output



Figure 8: Output of Program 12

GitHub Scan Code



Scan to view source code
[cite: 49]

Program 13:
Write a program to Encapsulated Banking System with Inheritance Models.

Code

```
1
2  class BankAccount:
3      def __init__(self, account_no, balance):
4          self.__account_no = account_no
5          self.__balance = balance
6
7      def deposit(self, amount):
8          if amount > 0:
9              self.__balance += amount
10             print("Amount deposited:", amount)
11         else:
12             print("Invalid amount.")
13
14     def withdraw(self, amount):
15         if 0 < amount <= self.__balance:
16             self.__balance -= amount
17             print("Amount withdrawn:", amount)
18         else:
19             print("Insufficient balance.")
20
21     def get_balance(self):
22         return self.__balance
23
24     def display(self):
25         print("Account Number:", self.__account_no)
26         print("Balance:", self.__balance)
27
28
29 class SavingsAccount(BankAccount):
30     def add_interest(self, rate):
31         interest = self.get_balance() * rate / 100
32         self.deposit(interest)
33         print("Interest added:", interest)
34
35
36 class CurrentAccount(BankAccount):
37     def withdraw(self, amount):
38         if amount <= self.get_balance() + 5000:
39             # Allows overdraft up to 5000
40             super().withdraw(amount)
41         else:
42             print("Overdraft limit exceeded.")
43
44
45 # Savings account
46 print("----- Savings Account -----")
47
48 savings = SavingsAccount("S101", 10000)
49
50 savings.display()
```

```
savings.deposit(2000)
savings.withdraw(1000)
savings.add_interest(5)

savings.display()


# Current account
print("\n----- Current Account -----")

current = CurrentAccount("C101", 5000)

current.display()
current.deposit(3000)
current.withdraw(7000)

current.display()
```

0.0.9 Output



Figure 9: Output of Program 13

GitHub Scan Code

Scan to view source code
[cite: 49]

Program 14:
Write a program for Object Persistence Engine via Serialization.

Code

```
1
2  import pickle
3  import shelve
4
5
6  class Student:
7      def __init__(self, name, roll_no, marks):
8          self.name = name
9          self.roll_no = roll_no
10         self.marks = marks
11
12     def display(self):
13         print("Name:", self.name)
14         print("Roll No:", self.roll_no)
15         print("Marks:", self.marks)
16
17
18 # Create object
19 student = Student("Priya", 101, 85)
20
21 # Pickle serialization
22 with open("student.pkl", "wb") as file:
23     pickle.dump(student, file)
24
25 print("Object saved using pickle.")
26
27
28 # Read object using pickle
29 with open("student.pkl", "rb") as file:
30     restored_student = pickle.load(file)
31
32 print("\nRestored object:")
33 restored_student.display()
34
35
36 # Store object using shelve
37 with shelve.open("student_data") as db:
38     db["student1"] = student
39
40 print("\nObject saved using shelve.")
41
42
43 # Read object from shelve
44 with shelve.open("student_data") as db:
45     restored = db["student1"]
46
47 print("\nObject retrieved from shelve:")
48 restored.display()
```

0.0.10 Output

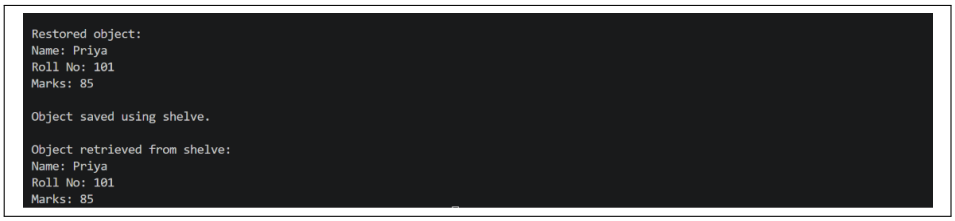


Figure 10: Output of Program 14

GitHub Scan Code

Scan to view source code
[cite: 49]

Program 15:
Write a program to Abstract Shape Math Engine with Operator Overloading.

Code

```
from abc import ABC, abstractmethod


class Shape(ABC):

    @abstractmethod
    def area(self):
        pass


class Vector(Shape):

    def __init__(self, x, y):
        self.x = x
        self.y = y

    def area(self):
        return abs(self.x * self.y)

    # Operator +
    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    # Operator ==
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    # Operator <
    def __lt__(self, other):
        return self.area() < other.area()

    # Operator string representation
    def __str__(self):
        return f"({self.x}, {self.y})"


# Create objects
v1 = Vector(3, 4)
v2 = Vector(2, 5)

# Addition
v3 = v1 + v2

print("Vector 1:", v1)
print("Vector 2:", v2)
print("Addition:", v3)

# Area
print("Area of Vector 1:", v1.area())
```

```python
print("Area of Vector 2:", v2.area())

# Comparison
print("Are vectors equal?", v1 == v2)
print("Is Vector 1 smaller?", v1 < v2)
```

0.0.11 Output



Figure 11: Output of Program 15

GitHub Scan Code

Scan to view source code
[cite: 49]

Program 16:
Write a program to create a Safe File Processor with Custom Exception Hierarchy.

Code

```
1
2  class FileProcessorError(Exception):
3      pass
4
5
6  class FileNotFoundErrorCustom(FileProcessorError):
7      pass
8
9
10 class InvalidFileError(FileProcessorError):
11     pass
12
13
14 def process_file(filename):
15     try:
16         if not filename.endswith(".txt"):
17             raise InvalidFileError("Only .txt files are allowed.")
18
19         try:
20             file = open(filename, "r")
21         except FileNotFoundError:
22             raise FileNotFoundErrorCustom("File does not exist.")
23
24         with file:
25             content = file.read()
26
27             if content.strip() == "":
28                 raise InvalidFileError("File is empty.")
29
30             print("\nFile Content:")
31             print(content)
32
33     except FileProcessorError as e:
34         print("Error:", e)
35
36     else:
37         print("\nFile processed successfully.")
38
39     finally:
40         print("File processing completed.")
41
42
43 filename = input("Enter text file name: ")
44 process_file(filename)
```

0.0.12 Output

```
File Content:
lorem435
Python is a high-level programming language known for its simple and readable syntax. It has the following feat
ures:

Allows writing clean code with fewer lines.
Supports multiple programming paradigms including object-oriented, functional and procedural programming.
Widely used in web development, automation, data analysis, artificial intelligence and many other fields.
Dynamically typed and has automatic garbage collection


File processed successfully.
File processing completed.
```

Figure 12: Output of Program 16

GitHub Scan Code

Scan to view source code
[cite: 49]

Program 17:
Write a program to create a Multi-Functional Graphical Desktop Calculator.

Code

```
1
2  import tkinter as tk
3
4
5  def button_click(value):
6      current = display.get()
7      display.delete(0, tk.END)
8      display.insert(0, current + value)
9
10
11 def calculate():
12     try:
13         result = eval(display.get())
14         display.delete(0, tk.END)
15         display.insert(0, str(result))
16     except:
17         display.delete(0, tk.END)
18         display.insert(0, "Error")
19
20
21 def clear():
22     display.delete(0, tk.END)
23
24
25 root = tk.Tk()
26 root.title("Desktop Calculator")
27 root.geometry("300x400")
28
29 display = tk.Entry(
30     root,
31     font=("Arial", 20),
32     justify="right"
33 )
34 display.grid(row=0, column=0, columnspan=4,
35              padx=10, pady=10)
36
37
38 buttons = [
39     ("7", 1, 0), ("8", 1, 1), ("9", 1, 2), ("/", 1, 3),
40     ("4", 2, 0), ("5", 2, 1), ("6", 2, 2), ("*", 2, 3),
41     ("1", 3, 0), ("2", 3, 1), ("3", 3, 2), ("-", 3, 3),
42     ("0", 4, 0), (".", 4, 1), ("+", 4, 2)
43 ]
44
45
46 for text, row, column in buttons:
47     tk.Button(
48         root,
49         text=text,
50         width=6,
```

```
        height=2,
        command=lambda x=text: button_click(x)
    ).grid(row=row, column=column, padx=5, pady=5)


tk.Button(
    root,
    text="=",
    width=6,
    height=2,
    command=calculate
).grid(row=4, column=3, padx=5, pady=5)


tk.Button(
    root,
    text="Clear",
    width=25,
    height=2,
    command=clear
).grid(row=5, column=0, columnspan=4, pady=10)


root.mainloop()
```

0.0.13 Output

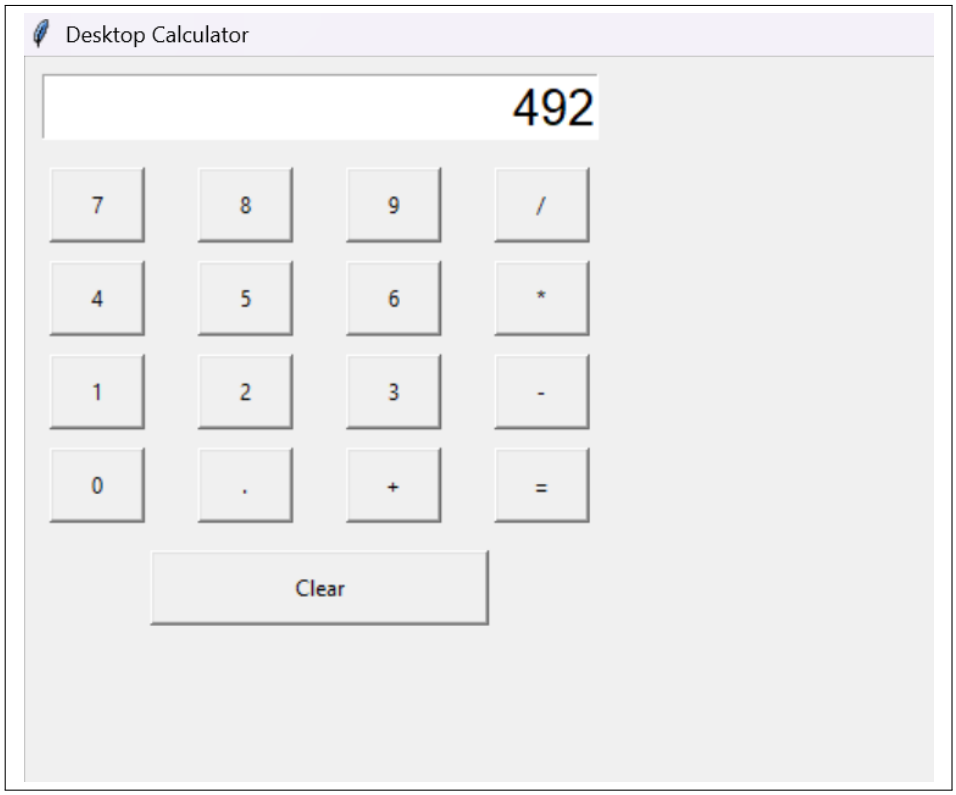


Figure 13: Output of Program 17

GitHub Scan Code



Scan to view source code
[cite: 49]

Program 18:
Write a program to create a GUI-based student record manager that stores student details in a file.

Code

```
1
2  import tkinter as tk
3  from tkinter import messagebox
4
5
6  def save_record():
7      name = name_entry.get()
8      roll = roll_entry.get()
9      course = course_entry.get()
10     gender = gender_var.get()
11
12     hobbies = []
13
14     if reading_var.get():
15         hobbies.append("Reading")
16
17     if sports_var.get():
18         hobbies.append("Sports")
19
20     if music_var.get():
21         hobbies.append("Music")
22
23     address = address_text.get("1.0", tk.END).strip()
24
25     if name == "" or roll == "" or course == "":
26         messagebox.showerror("Error", "Please fill all required
       fields.")
27         return
28
29     with open("student_records.txt", "a") as file:
30         file.write("Name: " + name + "\n")
31         file.write("Roll No: " + roll + "\n")
32         file.write("Course: " + course + "\n")
33         file.write("Gender: " + gender + "\n")
34         file.write("Hobbies: " + ", ".join(hobbies) + "\n")
35         file.write("Address: " + address + "\n")
36         file.write("-" * 40 + "\n")
37
38     messagebox.showinfo("Success", "Student record saved
       successfully.")
39
40
41 def clear_fields():
42     name_entry.delete(0, tk.END)
43     roll_entry.delete(0, tk.END)
44     course_entry.delete(0, tk.END)
45     address_text.delete("1.0", tk.END)
46
```

```
    gender_var.set("Male")
    reading_var.set(False)
    sports_var.set(False)
    music_var.set(False)


root = tk.Tk()
root.title("Student Record Manager")
root.geometry("400x500")

tk.Label(root, text="Student Record Manager",
         font=("Arial", 18)).pack(pady=10)

tk.Label(root, text="Name").pack()
name_entry = tk.Entry(root, width=40)
name_entry.pack()

tk.Label(root, text="Roll Number").pack()
roll_entry = tk.Entry(root, width=40)
roll_entry.pack()

tk.Label(root, text="Course").pack()
course_entry = tk.Entry(root, width=40)
course_entry.pack()

tk.Label(root, text="Gender").pack()

gender_var = tk.StringVar(value="Male")

tk.Radiobutton(root, text="Male",
               variable=gender_var,
               value="Male").pack()

tk.Radiobutton(root, text="Female",
               variable=gender_var,
               value="Female").pack()

tk.Label(root, text="Hobbies").pack()

reading_var = tk.BooleanVar()
sports_var = tk.BooleanVar()
music_var = tk.BooleanVar()

tk.Checkbutton(root, text="Reading",
               variable=reading_var).pack()

tk.Checkbutton(root, text="Sports",
               variable=sports_var).pack()

tk.Checkbutton(root, text="Music",
               variable=music_var).pack()

tk.Label(root, text="Address").pack()

address_text = tk.Text(root, width=35, height=4)
address_text.pack()

tk.Button(root, text="Save Record",
```

```
            command=save_record).pack(pady=10)

tk.Button(root, text="Clear",
            command=clear_fields).pack()

root.mainloop()
```

0.0.14 Output

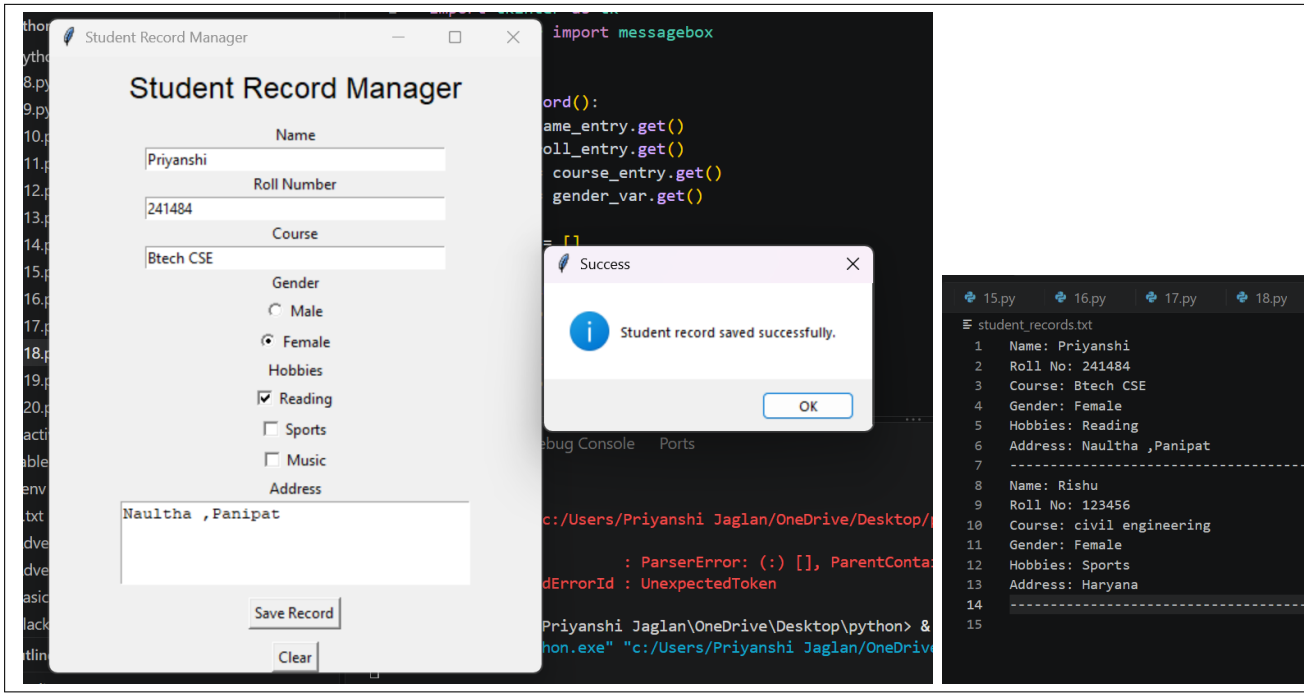


Figure 14: Output of Program 18

GitHub Scan Code

Scan to view source code
[cite: 49]

Program 19:
Write a program to create a GUI application that allows the user to select and view image or text files using dialog boxes.

Code

```
import tkinter as tk
from tkinter import filedialog, messagebox
from PIL import Image, ImageTk


def open_file():
    global photo

    filename = filedialog.askopenfilename(
        title="Select File",
        filetypes=[
            ("Text Files", "*.txt"),
            ("Image Files", "*.png *.jpg *.jpeg"),
            ("All Files", "*.*")
        ]
    )

    if filename == "":
        return

    try:
        if filename.lower().endswith(".txt"):
            with open(filename, "r") as file:
                content = file.read()

            text_area.delete("1.0", tk.END)
            text_area.insert(tk.END, content)

            image_label.config(image="")
            file_label.config(text="Text File: " + filename)

        else:
            image = Image.open(filename)
            image.thumbnail((500, 350))

            photo = ImageTk.PhotoImage(image)

            image_label.config(image=photo)
            text_area.delete("1.0", tk.END)

            file_label.config(text="Image File: " + filename)

    except Exception as e:
        messagebox.showerror("Error", str(e))


root = tk.Tk()
```

```
root.title("Image / File Viewer")
root.geometry("650x550")

tk.Label(
    root,
    text="Dynamic File and Image Viewer",
    font=("Arial", 18)
).pack(pady=10)

tk.Button(
    root,
    text="Open File",
    command=open_file
).pack(pady=10)

file_label = tk.Label(root, text="No file selected")
file_label.pack()

text_area = tk.Text(root, width=70, height=15)
text_area.pack(pady=10)

image_label = tk.Label(root)
image_label.pack()

root.mainloop()
```

0.0.15 Output

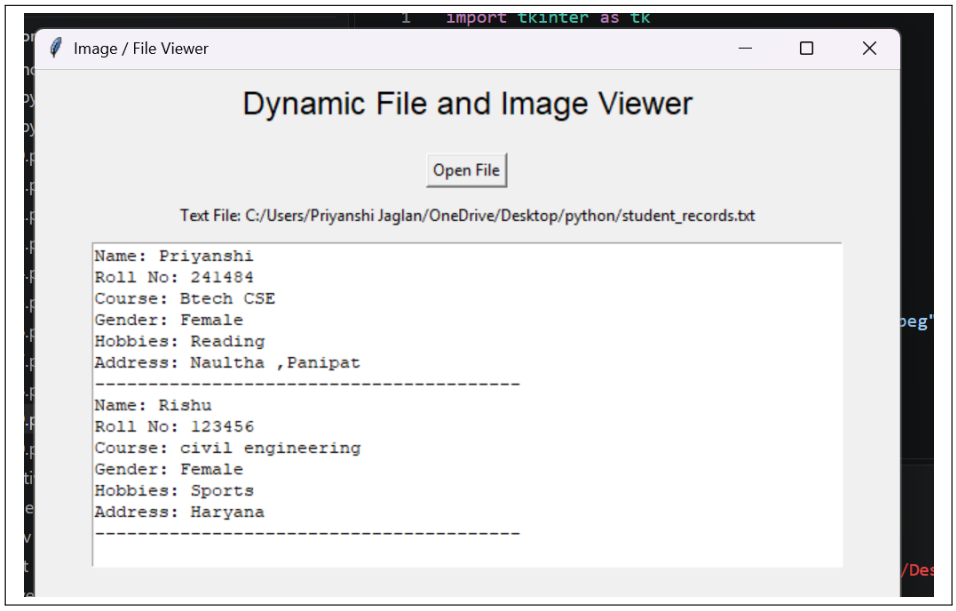


Figure 15: Output of Program 19

GitHub Scan Code



Scan to view source code
[cite: 49]

Program 20:
Write a program to create a modular multi-frame GUI dashboard using Tkinter, menus, buttons and frames.

Code

```
1
2  import tkinter as tk
3  from tkinter import messagebox
4
5
6  def show_frame(frame):
7      frame.tkraise()
8
9
10 def save_data():
11     with open("dashboard_data.txt", "w") as file:
12         file.write("Dashboard configuration saved successfully.\n")
13
14     messagebox.showinfo(
15         "Success",
16         "Dashboard data saved successfully."
17     )
18
19
20 root = tk.Tk()
21 root.title("Multi-Frame Dashboard")
22 root.geometry("700x450")
23
24
25 # Create container
26 container = tk.Frame(root)
27 container.pack(fill="both", expand=True)
28
29 container.grid_rowconfigure(0, weight=1)
30 container.grid_columnconfigure(0, weight=1)
31
32
33 # Create frames
34 home_frame = tk.Frame(container)
35 student_frame = tk.Frame(container)
36 settings_frame = tk.Frame(container)
37
38
39 for frame in (home_frame, student_frame, settings_frame):
40     frame.grid(row=0, column=0, sticky="nsew")
41
42
43 # ---------------- HOME FRAME ----------------
44
45 tk.Label(
46     home_frame,
47     text="Welcome to Dashboard",
48     font=("Arial", 24)
```

```
).pack(pady=30)

tk.Label(
    home_frame,
    text="This is the Home Page"
).pack(pady=10)


# ---------------- STUDENT FRAME ----------------

tk.Label(
    student_frame,
    text="Student Information",
    font=("Arial", 22)
).pack(pady=20)

tk.Label(
    student_frame,
    text="Name: Student"
).pack(pady=5)

tk.Label(
    student_frame,
    text="Course: B.Tech CSE"
).pack(pady=5)

tk.Label(
    student_frame,
    text="Semester: 4"
).pack(pady=5)


# ---------------- SETTINGS FRAME ----------------

tk.Label(
    settings_frame,
    text="Settings",
    font=("Arial", 22)
).pack(pady=20)

tk.Label(
    settings_frame,
    text="Dashboard Settings"
).pack(pady=10)

tk.Button(
    settings_frame,
    text="Save Configuration",
    command=save_data
).pack(pady=10)


# ---------------- MENU BAR ----------------

menu_bar = tk.Menu(root)

home_menu = tk.Menu(menu_bar, tearoff=0)
home_menu.add_command(
```

```
    label="Home",
    command=lambda: show_frame(home_frame)
)
menu_bar.add_cascade(label="Home", menu=home_menu)


student_menu = tk.Menu(menu_bar, tearoff=0)
student_menu.add_command(
    label="Student",
    command=lambda: show_frame(student_frame)
)
menu_bar.add_cascade(label="Student", menu=student_menu)


settings_menu = tk.Menu(menu_bar, tearoff=0)
settings_menu.add_command(
    label="Settings",
    command=lambda: show_frame(settings_frame)
)
menu_bar.add_cascade(label="Settings", menu=settings_menu)


root.config(menu=menu_bar)


# Navigation buttons
navigation = tk.Frame(root)
navigation.pack(side="bottom", pady=10)

tk.Button(
    navigation,
    text="Home",
    command=lambda: show_frame(home_frame)
).pack(side="left", padx=5)

tk.Button(
    navigation,
    text="Student",
    command=lambda: show_frame(student_frame)
).pack(side="left", padx=5)

tk.Button(
    navigation,
    text="Settings",
    command=lambda: show_frame(settings_frame)
).pack(side="left", padx=5)


show_frame(home_frame)

root.mainloop()
```

0.0.16 Output

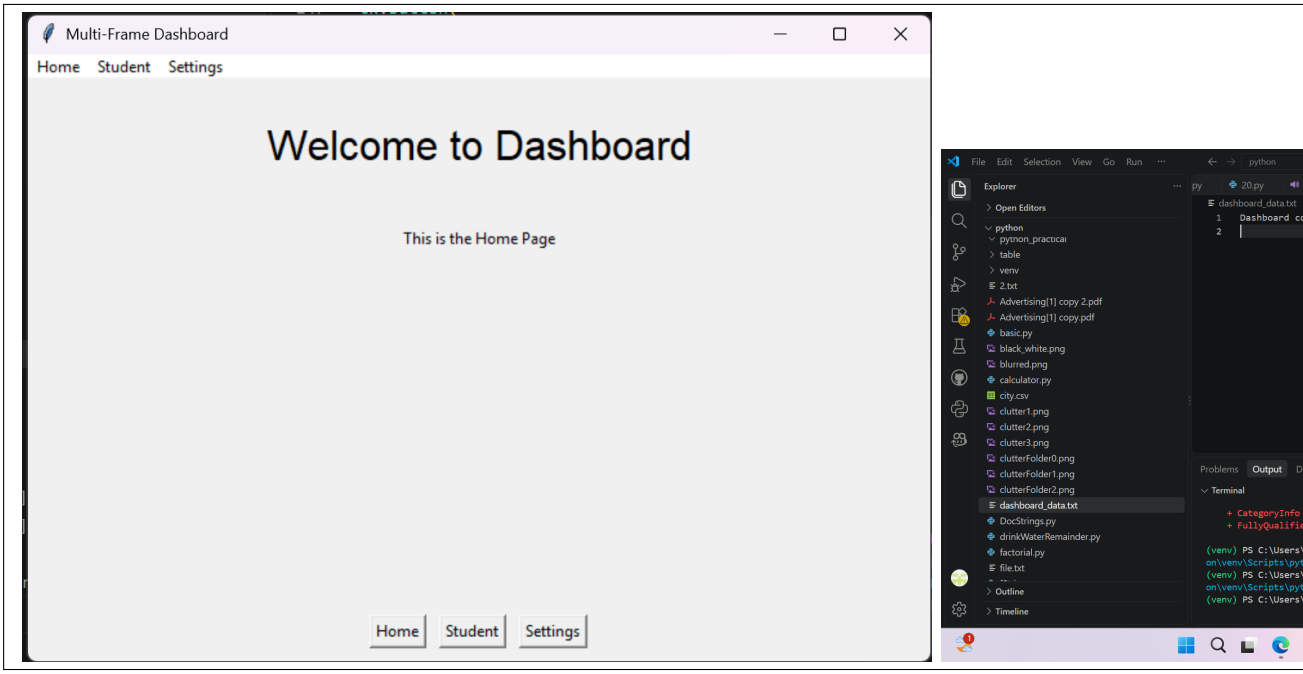


Figure 16: Output of Program 20

GitHub Scan Code

Scan to view source code
[cite: 49]

Lab Work Repository

Scan for Complete Lab Folder [cite: 49]